# Aligning Generative AI with Business Cases

## Evaluating and Leveraging Generative AI

### Intro and What Does Gen AI Mean for Developers?

Hi, my name is Jamie Maguire, and welcome to this course, Aligning Generative AI with Business Cases. I'm a software architect, developer, and Microsoft MVP in artificial intelligence. This demo-rich course shows you how to evaluate and leverage generative AI. This course will also teach you about the latest AI terms, development practices, and show you the art of the possible. Are you ready? Let's get stuck into the course. Throughout this course, we will see how to evaluate and leverage generative AI. We'll look at what's involved in creating an agent. We'll see how you can extend an AI agent's intelligence. And finally, you'll learn how to improve an AI agent's knowledge by referencing contextual and accurate data. So, what does generative AI mean for developers? It's a great time to be a developer. Recent advancements in AI have been incredible to see unfold, especially in these last 24 months. The use of generative AI is helping developers optimize the development workflow. The introduction of large language models to perform on demand and preemptive code generation is accelerating the development process. The less keystrokes, the better in my opinion. Generative AI is also a useful pair programmer. No more submitting questions to development forums and waiting for a response. Language models trained on vast quantities of global data can provide instant feedback. Using natural language when interacting with generative AI models, tools, and systems makes it easier for developers to express problems to the machine. A quicker feedback loop empowers developers and helps them solve problems quicker. From a greenfield development perspective, generative AI can help product teams or citizen developers rapidly create new prototypes. One of the most valuable features that generative AI offers to developers is the ability to quickly generate boilerplate code. Supplying a concise set of prompts or comments within your development ID is often enough for a language model to produce 50 to 60% of the required boilerplate code. It's not just boilerplate code, however. If you can accurately express a feature and the context in which it must be used in, you can also use generative AI to implement this. Recently, I had to code the settings screen within a .NET Core web application. The settings screen allowed users to supply an Azure API endpoint and its associated key. An HTML view was required, several input fields, and business logic. The view had to be wired up to a .NET controller. JavaScript was also required to let users toggle the API key field visibility. I expressed these requirements in a single prompt and asked the generative AI model to produce the relevant HTML, JavaScript, and C#

controller. The language model generated 70 to 80% of what was required to implement this feature, thereby saving me hours of development effort. The generation of test data is another area where generative AI can be useful. On a recent project that involved Elasticsearch, Semantic Kernel, and the .NET web API, I had to run several versions of queries against the raw Elasticsearch instance. Instead of coding each combination by hand, I instructed a language model to generate relevant query combinations with associated parameters. Generative AI can serve as a helpful real-time assistant and pair programmer for developers. It can provide instance suggestions, code snippets or explanations. We all have our strengths and weaknesses. In some of my technically weaker areas or unfamiliar codebases, I will ask GitHub Copilot what a block of code does. Code reviews are typically a manual process. GitHub Copilot can enhance developer code quality by offering automated code reviews. Another useful feature of generative AI when debugging code is being able to instantly ask an agent within your development ID what a particular error means. Automatic generation of code documentation is another helpful application. You may have already supplied a prompt to generate a series of automated unit tests for your class library. The automation doesn't end there though. You can ask the same model to then subsequently create a markdown file documenting the code that was automatically generated. Developers often must navigate hundreds of lines of code within a solution. It can be difficult to piece together the orchestration from one component to the next. You can use Gen AI to produce technical summaries of an existing codebase. Information assets produced by generative AI such as code documentation, technical summaries of API definitions can be ingested into an information store such as Azure Search or Elasticsearch. These assets can then be used as part of an internal knowledge base.

## Identifying and Prioritizing Suitable Use Cases

Let's take a look at how you can identify and prioritize use cases. You want to prioritize use cases that align closely with key business objectives or pain points. This might be use cases that can help solve current inefficiencies, improve product quality or enhance customer experience. Take into account both the tangible and intangible benefits. As part of this, you want to determine some evaluation metrics. Cost savings will likely be a key metric that you want to evaluate. Cost savings may come from reduced labor, process optimization or another may be to enhance decision-making. Take each of these into account. Compare these against the cost of what you're doing right now versus the cost of an AI approach. Increased revenue is likely another key evaluation metric. This might come from improved customer satisfaction or boosting conversion rates or possibly market expansion. Use generative AI to detect patterns in customer correspondence and reply accordingly. Augmenting business processes with intelligent models can help your business surface

patterns in customer behavior. Use this intelligence to boost conversion rates. Risk is another evaluation metric that you want to consider. Generative AI can help detect fraudulent patterns and anomalies by analyzing large datasets. Consider if the introduction of generative AI aligns with organizational goals and responsible AI principles. Use generative AI to enforce current regulatory processes and industry standards. Generative AI can help you measure and mitigate customer churn. By automating the detection of trends and providing predictive analytics, generative AI enables businesses to proactively address potential issues and optimize customer experiences. Evaluating the return of investment when implementing a generative AI solution is best achieved by implementing a pilot program. First, you'll want to identify suitable use cases within your organization. You can do this by performing traditional business analysis, doing an inventory of any data sources within your organization or by identifying any existing pain points or processes and workflows. You might even start by interacting with a language model explaining some of your current challenges and asking the model for suggestions. Ensure any use cases are well defined and focus on a specific problem or opportunity that align with organizational needs. Go for the low hanging fruit initially. Establishing clear goals provide a roadmap for success. Outline what the pilot program aims to achieve and set expectations for key stakeholders. Detail measurable outcomes. These let you assess the impact of your pilot's effectiveness. Evaluation metrics will form part of this and help you generate quantifiable data to evaluate the ROI of your pilot. These will affect scaling decisions. Start small to minimize risk and resource management. Gather data throughout your pilot program. Adopt an iterative approach to allow for refinement before scaling agenerative AI solution across broader use cases.

## Tools to Implement Gen AI

Let's look at some tools and services you can use to implement generative AI. Some tools you can implement generative AI solutions include Azure AI Foundry, Azure AI Services. GitHub Copilot, and Semantic Kernel. Azure AI Foundry is a new product from Microsoft. The product is an end-to-end solution for developing, deploying, and optimizing generative AI models that can be tailored to business needs. It ships with features such as custom model development, and you can also bring your own data too. It gives you access to the main AI models and additional AI services. Azure AI Foundry also ships with built-in governance and compliance tools, thereby ensuring responsible AI practices can be enforced. Azure AI services are an umbrella term for many AI tools. They ship with access to small and large language models. AI services can be accessed using containers, SDKs, and APIs. They let the machine read, comprehend, listen, search or process data and more. The services let you augment existing software with AI capabilities. Personally, these are some of my favorite generative AI tools, and I've used them on multiple projects. GitHub

Copilot, powered by generative AI, assist developers by producing automated code suggestions, completing functions, and automating repetitive tasks in real time. Copilot, when paired with Microsoft Graph connectors, let you create integrations with Microsoft 365 applications such as Teams, Outlook, and Word. Use these to create automation workflows and implement your user experiences. It doesn't always get it right, however, but my personal experience is that Copilot reduces the need to remember obscure syntax when developing in Visual Studio. It frees you up to focus on innovations and complex problem solving. Semantic Kernel is an open-source SDK designed to simplify the creation of agents that can interface with existing code. The SDK is extensible and supports models from OpenAI, Azure OpenAI, Hugging Face and more. The SDK lets you rapidly build agents and features language model integrations. Using the SDK means that you don't have to learn the inner workings of LLM providers, APIs or handle low-level function calling. Other tools that you can use to help with your generative AI journey include Audio Notes, Google Gemini, Midjourney, and Toolhouse. Audio Notes lets you create concise notes from the spoken words. It can also transcribe video content and produce summarized versions of the transcript. Google Gemini is a model that can handle multi-model inputs. It ships with features such as text generation, image creation, and complex data analysis. Midjourney is a platform focused on creating images. The tool can be used by founders, designers, marketers, and creators to generate content for projects, branding, and storytelling. Toolhouse is a platform designed to enhance and simplify LLM integration for developers. It lets developers focus on building agents without the overhead of managing complex integrations. Some final things to consider from this module are start small with the generative AI implementation, implement the bare minimum, gather data throughout your pilot program, and iterate.

# Creating Agents Using Semantic Kernel and Open AI ChatCompletions API

## Introduction

In this module, you'll learn how to create an AI agent. The demo will show you how to create an AI agent using Semantic Kernel with the OpenAI Chat Completions API.

## Demo: Creating an Agent

So, before we dive into the coding section of this demo, the first thing that you are going to need is a set of OpenAI developer keys, and you can get your own API keys by visiting platform.openai.com. After obtaining your keys, you can

proceed with the coding part of this demo. So we're in Visual Studio, and what we're looking at here is a simple console application. One of the first things that we have to do is to bring in two namespaces that belong to the Semantic Kernel namespace. So we'll do that just now. We can see that we have our API key within the region at line 10, and we are specifying on line 16 that we want to use the gpt-4 model from OpenAI. So our console application doesn't do much at the moment. And we can see we've got Copilot here actually trying to prompt us and help us with some suggestions, but we're not going to use those for just now. The first thing that we have to do is to create a kernel, and we can do that with this code here. What this does is it will create a kernel, which is responsible for the orchestration of the interaction with the language model, and we're telling the kernel that we want to include the OpenAIChatCompletion's endpoint, passing in the modelId and apiKey that we have defined earlier. On line 24, this is actually what we need to do. So we have to build this kernel. We do not want to use the GetChatCompletion's method at the moment because what I want to show is how we can also add a meta prompt to the agent, and we can do that by creating an entry into the chat history. So we have got some code here that does this. So the chat history is an object that you can add messages to from the human or the assistant that you're building. Okay, so with the kernel built and the chat history set up, what we can now start to do is create a chat session. Okay, so we will accept this comment from Copilot, and we won't accept this code at the moment. We need to create an instance of the chatCompletionService. First of all, we'll start the chat session. And what we have to do here is we will prompt the human for a question. What we'll do is we will introduce a while loop, and I've already got this code typed here, so we'll take a copy of this, and I'll explain what it does. So, first of all, we have to correct this error. So we've got a variable to store the user input. We have a compile time error here. And we need to make this console application asynchronous, so that fixes that. So what we can see here is we've got a while loop that runs until infinity. The first thing that we do is on line 40 after capturing the user's input is we add it to the history object. And between lines 43 and 45, we invoke the chatCompletionService. Okay, we pass in the history object that we have set up. So first time in, it's only going to have the meta prompt and the initial user message. And then we also pass in the kernel object. In line 48, we output the response from the assistant. And then on line 51, we add the assistant's reply to the history object, and then we just go back around the loop again prompting the user for another message. Okay, very simple, bare minimal, and it shows how we can in just a few lines of code connect to an LLM via Semantic Kernel. So we'll save this console application, and we will run it. So we can see here our application is running. I'll just make the text a bit bigger. And you can see our assistant has replied. Now, it came back with a jovial response, okay. And we can go through this interaction as many times as we want, go backwards and forwards. We can change the tonality of this agent by adjusting the meta prompt that we've seen earlier so we can stop the application. And do this, run the application again, and we get a very straightforward response. So that was a very quick runthrough of how we can use

Semantic Kernel to create an integration with language models hosted within OpenAI.

# Extending Agentic Behavior with Plugins and Native Functions

## Introduction

In this module, you'll learn about core concepts in Semantic Kernel. Automatic function calling, plugins, and native functions are also covered. We'll see how these can be used to extend an agent's behavior.

## Core Concepts in Semantic Kernel

At a high level, the kernel consists of three main components. The kernel acts as a core orchestrator and is responsible for managing all interactions between AI models, connectors, and any functions. AI connectors facilitate the integration with AI models such as Azure OpenAI, Phi-3 or any other custom models that you may have developed. AI connectors provide a common abstraction and interface for text generation and embeddings or any other AI tasks. Regardless of the underlying model that you integrate, AI connectors provide you with a common development experience. Data connectors let you interact with any underlying databases that are required by your application. These are typically vector databases. The kernel doesn't use any registered vector database out of the box, so it's your responsibility to define a data store. For rapid prototypes or proof of concepts, an in-memory data store can be used. Support for enterprise-grade vector databases such as Azure AI Search or Elasticsearch and others are available. At a high level, implementing a generative AI solution includes the following, the kernel, a series of connectors, memory, and multiple prompts. With the kernel being responsible for the orchestration of all components that interact with the language model, creating an intelligent agent grounded in your own data that can be instructed to follow your specific needs or business logic requires some additional components. Plugins encapsulate multiple prompts or business logic into a single reusable component. These can be developed in a language of your choice. Chances are you already have one or many class libraries within your existing software applications. You can repurpose any logic within a plugin. In a single line of code, a plugin can be made discoverable by the kernel. A plugin can contain one or many native functions. A native function is just a regular .NET method that has been decorated using natural language. You can decorate a native function using natural language to make it discoverable by the kernel and any language model. And here is the best part. You don't need to implement any of the orchestration logic to call your

plugins or native functions. The kernel, when integrated with a language model, will analyze and select the most relevant plugin to satisfy the user's current prompt. Automatic function calling helps make this possible. You can implement this by hand; however, Semantic Kernel shields you from implementing these low-level steps. It's worth understanding what's going on when function calling is in play. A user supplies the prompt that is sent to the AI model. The model analyzes the prompt to determine the user's intent. The model also extracts necessary parameters from the user prompt. The model checks if fulfilling the user's request requires an external function call. If a function call is required, the AI model identifies the appropriate function to call. The model calls the external function or API with the extracted parameters. The external function call processes requests and returns a response. The model uses the function response to generate a user-friendly reply. The model then sends the final response back to the user. If no function call is required, a normal response is returned by the model.

## Demo: Extending Agent Behavior

It's time for a demo. In this demo, you'll learn how to extend an agent's behavior with plugins and native functions. So in this demo, we are back in Visual Studio, and what we're going to do in this demo is we will build on the console application that we've seen in the earlier module; however, we're going to extend that to use plugins and native functions. So if we scroll down here, we can see we've just got the traditional agent that we had from earlier on. The code is the exact same. However, what we're going to do now is if you look over to the right-hand side, we've got a Plugins folder, okay? And the purpose of the agent in this situation is to help people with their health and fitness goals. And the way that it's going to do that is by offering up exercises. So if we open up the CardioPlugin, we can see here we've got a CardioPlugin that has two methods. And this is just a regular .NET class, and it's not doing anything at the minute. Okay, but we're going to extend that very quickly. We can open up the StrengthTrainingPlugin. And we can see here that we've got another method here. We'll just fix that typo. If we open up the program.cs file, which is the entry point for this agent, the first thing that we have to do is we have to make these plugins discoverable by Semantic Kernel. And the way that we do that is by referencing the builder object and using a method called autotype, so we'll do that just now. And we can see that Copilot has actually identified them for us almost. We just have to change the type. Okay. So that's the first one added in. We'll just copy and paste that, and we'll do the same for the StrengthTrainingPlugin. So the plugins have been made discoverable by the kernel and the LLM that we've got integrated. The next part of the puzzle is to set up automatic function calling. So we'll scroll down a little bit further here, and what we'll do is we will set that up here. So we're using OpenAI. And we need the OpenAIPromptExecutionSettings object. And the property that we want to set is

ToolCallBehavior. It's an enum. And what we do is we'll set that to AutoInvokeKernelFunctions, which is this method here. So, at this point, we have made the plugins discoverable. We've activated automatic function calling. The other thing that we have to do is pass the settings object into the chatCompletionService that we've got down here, so we'll do that. And at this point, we have everything set up to make the plugins discoverable and create a RAG-type experience. Now, the plugins themselves at the moment, they don't actually do very much, they just return an empty string. So, we have to decorate this method using natural language, and this is what helps the LLM and Semantic Kernel SDK discover the most relevant plugin and native functions for the particular prompt that's been issued. So rather than type it all in, what I'll do is I've already got it prepared. So I'm going to bring this over here and paste it in. And we'll just take a little moment just to run through what this is actually doing. So, this is a plugin with a single native function. And at line 16, we have the name of a native function. If we look at line 14 and 15, we can see we've got natural language to describe the purpose of this native function. We also have in line 17 a parameter that gets passed in. Now that's the muscle group to target for a particular exercise. So if you can imagine somebody interacting with their healthcare agent, the person might say, hey, I'm looking to target my chest, which exercises should I use? Well, what's going to happen is Semantic Kernel SDK coupled with the LLM are going to to discover this plugin and native function, and it's going to suggest an exercise, a series of exercises based on that muscle group. With that in mind, we'll do the similar thing for the CardioPlugin. Now this one's a little bit different. It doesn't accept any incoming parameters, but just for an overview at line 14, we describe what the purpose of this native function is, and it's to suggest low-impact cardio activities. And the key ones that we have are detailed between lines 17 and 23. Then at line 25, those activities get returned to Semantic Kernel and the LLM as part of the automatic function call. We have another native function that's responsible for suggesting regular cardio activities. So, this plugin handles two different types of activity. We have the StrengthTrainingPlugin that we'll open back up here. It accepts a parameter called muscleGroup and uses that to try and identify which exercises would be suitable for the person. Okay. So we've got everything in play now to run this agent, so we'll spin up this console application. And we'll run it. And now we can ask our agent for any low impact cardio activities. So the text doesn't have to match exactly what the natural language description of the method is, so we'll try this just now. Now at this point, the suitable plugin and native function are going to be identified with the intent that's being expressed in this prompt, and we should get a response back very quickly. Okay. Now we've got Walking, Swimming, Cycling, Rowing, Elliptical training, and then we've got a closeout statement here. So let's move this over to the side, and we'll open up the CardioPlugin. And if we take a look up here, we can try and pair each of these off, so we've got Walking, we have Swimming, we have Cycling, we have Rowing, and we have Elliptical training. Now, there isn't any supplementary information for each of these activities in our plugin, but that's

where the LLM has augmented the response from Semantic Kernel in the native function to create a more coherent and rich response. So this is retrieval augmented generation in action. Similarly, we can do the same for regular cardio activities. Again, we can pair them off so we can see at 1 we've got Running, we have Jumping rope, we have High-intensity or HIIT training, we have Aerobics, and we have Dancing. So that's also got supplementary information like before. And finally, we can try out the StrengthTrainingPlugin. If we pull this over here, we can see that this particular native function requires the parameter muscleGroup. So what we'll do this time is we will ask the agent for a particular exercise related to one of these muscle groups, okay? So we've got chest, back, shoulders, legs or arms. And again, just like before, the code will hit this particular plugin and native function. It will extract the exercises from 20 to 25, and it will use the parameter that gets passed in. Okay, so we can see in the response up here, we've got Pull-ups, Rows, and Deadlifts. And then if we look down here at line 22, we can see that those exercises are related to back strength conditioning. We can put a breakpoint on this particular native function and then inspect the parameter that gets passed in. And we'll just copy and paste in the prompt from earlier. And this breakpoint will get hit. If we hover over the muscleGroup parameter, we can see the SDK and LLM have been able to identify the particular muscle group that the person was to target. We can just skip past the breakpoint. And then we get the same answers back as before. So, that was it. In this demo, we have seen how to extend an agent using plugins and native functions.

# Improving Agent Knowledge with RAG, InMemory Vector Store, Open AI Embeddings API, and Semantic Kernel

## Introduction

In this module, you'll learn about core concepts in retrieval-augmented generation, or RAG. You'll learn how to apply these and build a RAG agent.

## Core Concepts in RAG

So, let's look at the core concepts that you can expect to find in RAG solutions. Retrieval-augmented generation, or RAG, blends generative AI content with existing information that you already have access to. This combination helps a language model produce enhanced responses with relevant and accurate data. An AI model grounded with existing information increases the probability of generating contextually rich answers to user prompts. The retriever is a core

component of RAG architecture. This may be implemented as a service class that can accept a series of inputs. The retriever makes relevant calls to obtain existing information to supplement the user prompt and models intelligence. Existing information can be fetched from many sources. This is additional information that a model has not yet been trained with, but you want to include this as part of processing. For example, you may want the model to include customer feedback from the last 30 days as part of a response. A generator uses retrieved information to produce responses. These AI components and processes ensure that AI systems can provide precise responses. When implementing RAG solutions, a mechanism is required that allows efficient processing, storing, and retrieval of data. This is where vector databases and embeddings come into play. Vector embeddings represent the semantic meaning of text and data. They are expressed in numerical format and are typically stored in databases such as Elasticsearch. Embeddings help capture the semantic similarity between data. This means your application can perform searches on data based on meaning. In essence, using vector embeddings to perform search operations lets you go beyond regular text search. So what does this look like? We can see the sample embedding for the string Hello World. The embeddings in this example have been truncated. You don't need to worry too much about manually generating embeddings, as many of the AI providers contain models that will do this for you. Embeddings generation SDKs or cloud services can be imported easily into your solution. A typical workflow for processing involves the following steps. A user or automated process will supply a prompt. The prompt will be converted into its embeddings representation. For example, the OpenAI text embeddings model, ada-002, can be used for this task. Vector data that matches closely with the input query is used by search algorithms to return the response. A popular search algorithm is cosine similarity. Cosine similarity is often used in natural language processing or recommendation systems. The algorithm measures the angle between two vectors. An output value of 1 implies that two vectors are the same, whereas a cosine similarity of 0 would indicate that there are no similarities between both vectors. Top ranked data is returned using similarity scores. The model combines retrieved data with generative AI capabilities to produce a relevant and accurate response.

## Demo: Implementing RAG

It's time to put these concepts into practice. In this demo, you'll learn how to improve an agent's knowledge using RAG. We'll use an in-memory vector store to persist the embeddings. To generate embeddings, we'll use the OpenAI embeddings generator. Semantic Kernel will bring all of this together. So, we are back in Visual Studio, and what we're looking at here is another console application. The first thing that we have to do, however, is we have to bring in another NuGet package that lets us connect to vector databases. So we'll do that just now. And because this NuGet package is in prerelease, we have to select this

checkbox. And this is the NuGet package that you're going to want. It's the Microsoft.SemanticKernel.Connectors.InMemory. It's in preview at the moment, so we will install that. Okay, we've got the NuGet package installed. So the purpose of this RAG agent is to give people exercise advice; however, what's different from this agent will be that rather than extract hard-coded data, we are going to fetch data from a RAG database, and there's a few things that we have to put in place to make this possible. We'll close this here. And if we look here, we can see that we've got a Plugins folder, Services, and VectorModel folder. I want to take your attention to the VectorModel folder. And what we have is an object to represent an exercise fact, and we can see that we have Key, Name, and Description. We'll scroll down a little bit further, and we have DescriptionEmbedding. Now it's the DescriptionEmbedding that will be the numerical representation of the actual exercise description, okay? What we can see though is that we've got some build errors, and we have to fix those first of all, so we'll do that just now. And the way that we can do that is by bringing in another namespace. We need to bring in the VectorData namespace, which will resolve the errors. So now that we've got a model to represent an exercise, we have the Key, the Name, the Description, and the DescriptionEmbedding, it can then be used within the application. The final thing worth noting about this is the decoration attributes that we can see on lines 15, 21, 27, and 33. The VectorStoreRecordKey indicates that this is the key for this particular record, whereas we've got VectorStoreRecordData for Name and Description, and we've got VectorStoreRecordData for the actual embedding itself. So we'll save this and close it. We can open the ExerciseHelper class. And the purpose of this class is to generate a list of exercise facts. And we can see that we have a fact for Deadlift with its associated description. We've got one for Bench Press, Squat, Pull-Up, and Overhead Press. So this is just a regular class that we can use to generate exercise facts. So now that we have a model and a helper to generate those items, we can take a look at the Services class. Now, the purpose of this class is to encapsulate all functionality that is required to interact with the vector database. And we can take a look at some of the methods that form those classes. And one of the first things to point out in this class is that we are using the text embeddings model ada-002. We can see that here. We've also got our API key, and what we've done here is we've taken these directly from the OpenAI developer platform. If you look down here, we can see that we have some compilation errors. These aren't actually compiler errors. If we hover over the error, we'll get another description, and we can see here it's because we're using an evaluation namespace and code. So we have to bring in another command to disable those, and the command that we need is this. And the errors in lines 25 and 26 are now gone. We can take a closer look at the method that we've got in this class called IngestDataIntoVectorStore. We'll just minimize the Solution Explorer. And what we can see is this method accepts the collection of exercise facts, and here we are creating the list of sample exercises from our Helper class. We are looping through them here, generating and embedding for the description for an exercise fact, and then here we simply insert the record into

the index. If it does exist, it doesn't update. So that's a quick overview of the vector store service. So we can save that again and just close this class down, and we'll open up the Solution Explorer again. So at this point, we run through what the VectorModel folder contains, the Services class, and finally, we'll take a look at the plugin. So here we are looking at the StrengthTrainingPlugin. I'm just going to close the Solution Explorer so we can see all of the code. And we will just bring this in. The purpose of this plugin is to suggest a strength training exercise when a person has supplied a prompt that wants to target a specific muscle group, and that's what we can see down in the native function at line 23. We've also got a bunch of compile time errors like we had before, but that's just because we have to bring in some namespaces and disable some of the warnings that we've got because we're using preview NuGet packages. So, we'll have a little bit of a closer look at some of the code before we resolve each of these errors. So here we have the constructor. We pass in InMemoryVectorStore and the OpenAITextEmbeddingGenerationService, as well as the chatCompletionService. So each of these objects are going to get passed in into the constructor and let us work with the vector database, generate embeddings, and also generate responses from the LLM. We can scroll down to the native function, and here we accept a parameter called muscleGroup. At line 27, we fetch a reference to the vector database. Before submitting a query to the vector database, we have to vectorize the search parameter, which in our instance is the muscleGroup, and that's what we can see here. With the muscleGroup vectorized, we're then able to pass that over into the vector database and perform a search. In our instance, we're only returning the top one result. After the results have been fetched, we then loop through each of the results of which there should only be one. After the results have been fetched, we add them to your list. We can scroll down a little bit further here. So by the time we get down to line 49, we have already ran a vector search, and this native function has to return a coherent response back to the caller. In our situation, the call is going to be a console application like we've used in other modules. We take the data that was fetched from the vector database installed into the variable results. We pull out the name and the description for the exercise that is best suited for that particular muscle group. That gets loaded into a variable called context. We then generate a prompt within the native function to send back to the LLM. So in this instance, we we're saying the user is looking for strength training exercises for the muscleGroup, which is the parameter that gets identified and passed in. We supplement the prompt by saying here are some relevant exercises from the database, and we plug that into the context. Context is a way for the LLM to reference memory, so we have the muscleGroup paired with the memory from the vector database. And then finally at the end of the prompt, we say provide a concise and helpful response based on the above information. When we've got the muscleGroup and the context of the agent memory set from our vector database, we can then call the chat completion's endpoint with that prompt and return that response back to the user. So, that's a runthrough of what this plugin and respective native function do. Before we close this, however, we need to

resolve these compile time errors, so we'll scroll back up. And we'll go through them, so we will just use Visual Studio to import the relevant namespaces. We can see that we've got a warning about this namespace only being suitable for evaluation purposes. We will bring in our code to disable that. That resolves this one. We'll go down to our native function. We'll bring in the Embeddings namespace and the VectorData namespace, and that's our plugin and native function good to go. So we'll save it again, close it, and we will go back over to the Solution Explorer. So at this point, we've covered the code in the vector model, we've discussed how the Service class is used, and we've covered the StrengthTrainingPlugin. We have to pull all this together for our agent, which will be a console application, so we'll do that by opening the program.cs file. So here we are looking at a bare bones console application, which will serve as our agent. And what we're using, just to recap, is the embeddings model, ada-002, to generate vector embeddings, and we're using the OpenAI gpt-4o model to generate chat completions, as well as two objects to house the embeddingService and the chatCompletionService. We initialize each of those here. However, we do have to bring in the code to disable the warnings that we've got. That's why we have the red error squiggles. So now that we've got the warnings disabled, we've got a reference to the models, and we've instantiated the embeddingService and the chat completions, we can start to fill out the code that will be used to help people interact with an agent that provides strength training exercises. And the first thing that we want to do is to create an in-memory vector store. The next thing that we have to do is to create a collection in this vector store that will house our ExerciseFact. Now that we've got a vector store and the collection has been set up within this vector store, we have to get some content in there, but before we do that, we have to actually call collection.CreateCollectionIfNotExists. We'll add some facts to the collection. Now, Copilot is trying to help us here, but we don't want to use that suggestion, so we'll escape that out because what we want to do is to use our vector service class to ingest data into this particular collection. Let's identify our method here, so we will accept this suggestion and this one here. So we'll add a friendly message to let us know when we can supply a prompt. So we will accept those suggestions. Now with the data store set up, the collection created and content ingested into the data store, we need to load and set up Semantic Kernel and help it discover the StrengthTrainingPlugin, so that's the next thing that we'll do. So we have to create the kernel itself. We have to add in the OpenAIChatCompletion service, passing in the completionsModelId and our apiKey. Next, we have to make the StrengthTrainingPlugin discoverable by the kernel. Now our StrengthTrainingPlugin does need a reference to the vectorStore, the embeddingService, and the chatCompletionService because when the plugin and native function are invoked, we also create a prompt and run a search against the vector database. With the plugin instantiated and the relevant parameters, it can then be attached to the builder, thereby making this plugin and its native function discoverable by Semantic Kernel. And we'll take Copilot's suggestion. We have to build the kernels. And we also have to define

the OpenAI prompt execution settings, and what we'll do here is we will set the ToolCallBehavior to auto invoke kernel functions. And what this does is it will let Semantic Kernel and the model automatically invoke our plugin and native function. (Working) And it's good practice to define a system prompt that instructs the agent how to behave. And we can do that by setting the initial message for the ChatHistory object. Now what we can do now is to start the conversational loop. And I already have the code at hand, which is very similar from our earlier module. We have to define a string for the user input. And we are good to go. So, what this loop does is it will wait for the user's input, it then gets added to the ChatHistory, and where the prompt is invoked is here between lines 75 and 79. When we get a response back from the model and the agent, that content is then added to the history, and that's what we can see on line 86. Very important to point out that when we supply a prompt to this particular agent, under the hood, Semantic Kernel and the model will identify the StrengthTrainingPlugin and extract out the relevant parameter. And our example is going to be a muscleGroup. That muscleGroup is then used as part of a vector query, which is then issued against vectorized content that was previously ingested using our exercise helper. Our response is then returned back to us. So we'll save this program. And we will run this agent. So the first thing that we can see is the sample data from our exercise helper has successfully been ingested into the vector database, and we've been prompted to issue a question. So we can say something like I'd like to target my back muscles, which exercises would you suggest? Now before we issue this query, it'll be good to see what's going on under the hood. So we'll put a breakpoint on the native function. And we will put a breakpoint here. We'll open the console application again, and when we press Return, our native function will get invoked by Semantic Kernel and the LLM. So, the native function has been invoked. We can hover over the muscleGroup. And we can see that the correct muscleGroup has been identified. We have results. And we scroll down here. We'll use this part of this prompt to return a more fleshed out and coherent response back to us, right? Now it would just return our single exercise, which is 1. We can expose this variable here. And we can see that it's got our ExerciseFact, which is Pull-Up along with the description. Okay. So let's just leave one part of the data that's going to be used to generate the response. The LLM will come up with its own exercises as well. This is RAG in action, so we'll press Continue, and the agent is now going to generate a response for us. So, we can see the agent says to target your back, consider incorporating the following exercises. So, we have Pull-Ups, which is the data from our vector database, Bent-Over Rows, Lat Pulldowns, Deadlifts, and then we've got a closeout statement here. So we've seen a few things happen here. We've seen our agent set up our example exercise facts and ingest them through the vector service. We have seen how we can issue a question to an agent, which is then vectorized, and the relevant parameters are extracted using the language model in Semantic Kernel. We saw how under the hood within the native function the data from the vector database is then used as part of another prompt within the native function to return a response to

us.